AI Podcast Generator

System Design Document

Automated Multi-Speaker Podcast Generation Pipeline

Using LLM Agents, TTS, and Audio Post-Processing

Author: Omkar

Project Type: Personal / Open Source

Version: 1.0 (Draft)

Date: March 2026

Table of Contents

1. Executive Summary

This document describes the complete system design for an AI-powered podcast generation pipeline. The system accepts a topic from the user, performs deep research across the web, structures the findings into chapters, generates natural multi-speaker dialogue complete with human-like speech patterns (fillers, interruptions, backchannels, false starts), synthesises the dialogue into audio using text-to-speech (TTS), and performs audio post-processing to produce a polished, ready-to-publish podcast episode of up to 30 minutes.

The core challenge this system solves is not just "generating speech from text" but rather creating a listening experience that feels genuinely conversational. Most existing podcast generators produce output that sounds like two chatbots reading scripts at each other. This system addresses that through a dedicated Naturalness Injector agent, SSML-based prosody control, and an audio overlap engine that physically layers speech tracks to simulate real cross-talk.

Target Outcome

A 15–30 minute podcast MP3 file with 2–3 distinct speakers, natural conversational flow, topic-grounded factual content, and professional audio quality — generated fully automatically from a single topic input.

1.1 Key Design Principles

- **Research-grounded:** Every claim made in the podcast must trace back to a real source document. The system never relies solely on LLM parametric knowledge for factual claims.
- **Human-like naturalness:** The dialogue must include the imperfections that make real conversations feel real — fillers, interruptions, false starts, backchannels, laughter, and varied pacing.
- **Modular pipeline:** Each stage is an independent agent or processing node. This allows individual components to be swapped (e.g., changing TTS provider) without rewriting the pipeline.
- **Chapter-parallel processing:** After content planning, each chapter flows through the dialogue → TTS → audio pipeline independently, enabling parallelism and keeping LLM context windows manageable.
- **Quality gates:** Multiple validation agents (fact-checker, QA reviewer, listener simulator) catch errors before they reach the expensive TTS stage.

2. System Architecture Overview

The system is organised into five sequential phases. Each phase contains one or more agents or processing nodes. Data flows top-to-bottom through these phases, with the output of one phase becoming the input to the next. Within Phase 4 and Phase 5, chapter-level parallelism is employed — meaning multiple chapters can be processed simultaneously since they are independent of each other.

2.1 Five-Phase Architecture

Phase	Name	Key Agents	Output
Phase 1	Research and Ingestion	Query Producer, Web Scraper, Dedup + Relevance Scorer	Top-K ranked source chunks
Phase 2	Content Planning	Chapter Planner, Character Designer	Chapter outlines + character persona JSONs
Phase 3	Dialogue Generation	Dialogue Engine, Naturalness Injector, SSML Annotator, Fact-Checker, QA Reviewer	SSML-annotated script per chapter
Phase 4	Voice Synthesis	TTS Router (per speaker)	Raw audio clips per utterance per chapter
Phase 5	Audio Post-Processing	Overlap Engine, Post-Processor, Chapter Stitcher, Cold Open Generator	Final podcast MP3

2.2 Data Flow Summary

The data transformations across the pipeline can be summarised as follows. Understanding what data shape enters and exits each node is critical for implementation, as each agent's system prompt and tool access depends on its input/output contract.

1. **Topic (string)** → Query Producer → 10 search queries (list of strings)
2. **10 queries** → Web Search + Extraction → ~100 raw page contents (list of text blocks)

3. **~100 raw pages** → Dedup + Relevance Scorer → Top-K ranked chunks (~50–80 chunks with scores)
4. **Ranked chunks** → Chapter Planner → N chapter outlines (structured JSON with titles, key points, source refs)
5. **Chapter outlines** → Character Designer → Character persona profiles (JSON with speaking style, expertise, quirks)
6. **Outline + Personas** → Dialogue Engine → Clean multi-turn script (structured dialogue JSON per chapter)
7. **Clean script** → Naturalness Injector → Annotated script with [INTERRUPT], [FILLER], [LAUGH] markers
8. **Annotated script** → Fact-Checker → Verified script (claims cross-referenced against source chunks)
9. **Verified script** → QA Reviewer → Final script (checked for repetition, flow, engagement)
10. **Final script** → SSML Annotator → SSML-tagged utterances per speaker per chapter
11. **SSML utterances** → TTS → Raw audio WAV files per utterance
12. **Raw WAVs** → Overlap Engine + Post-Processor → Mixed chapter audio files
13. **Chapter audio** → Stitcher + Cold Open → Final podcast MP3

3. Phase 1: Research and Ingestion

This phase is responsible for transforming a user's topic into a curated, deduplicated, relevance-ranked knowledge base of source material. This is the foundation of the entire podcast — if the research is poor, every downstream agent will produce poor output. The phase contains three nodes that execute sequentially.

3.1 Query Producer Agent

The Query Producer is a ReAct-style LLM agent whose sole job is to take the user's topic description and generate a diverse set of 10 search queries that will collectively cover the topic from multiple angles. This is important because a single search query will only surface one perspective on a topic. For a 30-minute podcast to be interesting, you need breadth.

3.1.1 Why 10 Queries?

Consider a topic like "The impact of AI on healthcare". A naive approach would be to search for exactly that phrase. But that misses specific sub-topics that make for compelling podcast segments: drug discovery acceleration, diagnostic imaging breakthroughs, hospital workflow automation, patient privacy concerns, regulatory challenges, real clinical trial results, failures and controversies, and expert opinions. Each of these deserves its own search query to surface the best content.

The Query Producer is instructed to generate queries across these dimensions:

- **Foundational/definitional:** "What is AI in healthcare" — gives background context
- **Recent developments:** "AI healthcare breakthroughs 2025 2026" — gives freshness
- **Controversy/criticism:** "AI healthcare risks failures" — gives tension for debate
- **Expert opinions:** "leading researchers AI medicine interviews" — gives authoritative quotes
- **Case studies:** "hospital AI implementation case study" — gives concrete examples
- **Future outlook:** "future of AI healthcare predictions" — gives forward-looking content
- **Adjacent domains:** "AI pharma drug discovery" or "AI mental health apps" — gives breadth

3.1.2 Agent Configuration

The Query Producer agent receives the following inputs and produces the following output:

Parameter	Value	Notes
Input	User topic string + optional description	The more detail the user provides, the better the queries will be
LLM Model	Claude Sonnet (fast tier)	Query generation is a low-reasoning task; speed matters more than depth here
System Prompt	"You are a research query strategist..."	Includes instructions for diversity, specificity, recency-bias
Output Format	JSON array of 10 strings	Each string is a Google-search-ready query
Temperature	0.7	Slightly creative to get diverse queries, not too wild

Parameter	Value	Notes
Fallback	If user provides their own queries, skip this agent	User override always takes priority

Implementation Note

If the user provides a very short topic (e.g., just "AI"), the agent should first ask the LLM to expand it into a 2–3 sentence description, then generate queries from that expanded description. This prevents overly generic queries.

3.2 Web Search and Content Extraction

For each of the 10 queries produced by the Query Producer, the system executes a Google search (via SerpAPI, Google Custom Search API, or similar) and retrieves the top 10 results per query. This yields approximately 100 URLs. The system then fetches and extracts the textual content from each URL.

3.2.1 Content Extraction Pipeline

Raw HTML pages contain enormous amounts of noise: navigation menus, ads, sidebars, footers, cookie banners, and JavaScript artefacts. The extraction pipeline must strip all of this and retain only the article body text. This is a non-trivial problem.

The recommended approach is a two-stage extraction:

14. **Stage 1 — HTML-to-text extraction:** Use a library like Trafilatura (Python) or Mozilla's Readability.js. These libraries are specifically designed to extract the "main content" from a web page, stripping navigation, ads, and boilerplate. Trafilatura is preferred for its higher accuracy on news articles and blog posts.
15. **Stage 2 — Quality filter:** After extraction, apply a minimum length filter (discard pages with fewer than 200 words of extracted text) and a language filter (discard non-English pages unless the topic requires them). Also discard pages that are primarily lists of links, product pages, or login walls.

3.2.2 Rate Limiting and Robustness

Fetching 100 pages in rapid succession will get your IP blocked by most websites. The system must implement polite scraping practices:

- **Rate limiting:** Maximum 2 requests per second across all domains, with a 1-second delay between requests to the same domain.
- **Retry logic:** 3 retries with exponential backoff for transient failures (429, 503, timeout).
- **User-agent rotation:** Use a realistic browser user-agent string to avoid bot detection.
- **Timeout:** 10-second timeout per page. If a page takes longer, skip it — there are 99 others.
- **Concurrent fetching:** Use asyncio (Python) with a semaphore of 5–10 to parallelise fetches while respecting rate limits.

3.3 Deduplication and Relevance Scoring

This is the first of the improvements over the original design and arguably the most important preprocessing step in the entire pipeline. Without it, the Chapter Planner receives ~100 raw documents that may contain massive amounts of redundancy (many articles saying the same thing), noise (irrelevant tangents), and contradiction (conflicting claims). The Dedup + Relevance Scorer cleans this up.

3.3.1 Step-by-Step Process

Step 1: Chunking. Split each of the ~100 extracted pages into overlapping chunks of 300–500 words. Use a sliding window with 50-word overlap to avoid cutting important context at chunk boundaries. This converts ~100 documents into ~500–1000 chunks. Why chunk? Because a single 3000-word article may contain one paragraph that's highly relevant and 20 paragraphs that aren't. Chunking lets us keep only the relevant paragraphs.

Step 2: Embedding Generation. Generate a dense vector embedding for each chunk using a lightweight sentence transformer model. Recommended models: all-MiniLM-L6-v2 (free, fast, 384-dimensional) for local/self-hosted, or Amazon Titan Embeddings on Bedrock if you want managed infrastructure. The embedding captures the semantic meaning of each chunk as a fixed-length vector. Two chunks that discuss the same topic in different words will have similar embeddings.

Step 3: Deduplication via Cosine Similarity. Compute pairwise cosine similarity between all chunk embeddings. For any pair of chunks where similarity exceeds a threshold (recommended: 0.85), collapse them into a single chunk by keeping the longer or more information-dense version. This is implemented efficiently using FAISS or scikit-learn's cosine_similarity on the embedding matrix. You don't need to compare every pair — use approximate nearest neighbour search (FAISS IVF) to find only the close pairs.

Step 4: Relevance Reranking. After deduplication, rank the surviving chunks by their relevance to the original user topic. Use a cross-encoder reranker model such as ms-marco-MiniLM-L-6-v2. A cross-encoder takes a (query, document) pair and outputs a relevance score, which is much more accurate than cosine similarity of independently-computed embeddings. The "query" here is the user's original topic description; the "document" is each surviving chunk.

Step 5: Top-K Selection. Sort chunks by their cross-encoder relevance score in descending order. Keep the top K chunks (recommended K = 50–80 for a 30-minute podcast). These are the chunks that will be passed to the Chapter Planner. Each chunk retains metadata: its source URL, original page title, and relevance score.

Why Not Just Use LLM Summarisation?

You might wonder: why not just feed all 100 pages to an LLM and ask it to summarise? Three reasons: (1) 100 pages is far too large for any context window, even Claude's 200K tokens. (2) Embedding-based dedup is deterministic and fast; LLM summarisation is slow and non-deterministic. (3) By keeping actual source chunks, you preserve the ability to trace every claim in the final podcast back to its original source — this is critical for the Fact-Checker agent later in the pipeline.

3.3.2 Technical Specifications

Component	Recommendation	Why
Chunking	LangChain RecursiveCharacterTextSplitter, chunk_size=500, overlap=50	Respects sentence boundaries, widely used in RAG pipelines
Embedding Model	all-MiniLM-L6-v2 (sentence-transformers) or Titan Embeddings	Fast, accurate for semantic similarity, runs locally on CPU

Component	Recommendation	Why
Similarity Threshold	0.85 cosine similarity	Below 0.85 catches most true duplicates without merging merely related content
Index	FAISS IndexFlatIP or IndexIVFFlat	Efficient nearest-neighbour search, no GPU needed for <1000 vectors
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2	Best accuracy-to-speed ratio for relevance scoring
Top-K	50–80 chunks	Enough material for 6–8 chapters without overwhelming the Chapter Planner

4. Phase 2: Content Planning

Phase 2 takes the ranked source chunks from Phase 1 and transforms them into a structured podcast plan: how many chapters, what each chapter covers, and who the speakers are. This phase contains two agents that work sequentially.

4.1 Chapter Planner Agent

The Chapter Planner receives the top-K ranked chunks and must organise them into a coherent narrative structure. This is fundamentally a clustering + sequencing problem: group related chunks into chapters, then order the chapters into a narrative arc that keeps listeners engaged for 30 minutes.

4.1.1 Narrative Arc Design

A good podcast episode follows a three-act structure, just like a story. The Chapter Planner must be instructed to create this arc:

16. **Act 1 — Setup (Chapters 1–2):** Introduce the topic, explain why it matters, provide necessary background. These chapters should be accessible to someone who knows nothing about the topic. The host "discovers" the topic alongside the listener.
17. **Act 2 — Exploration (Chapters 3–5):** Deep dives into specific sub-topics. This is where the experts (guest characters) share their knowledge, disagree with each other, and explore nuances. Each chapter should have a distinct angle or sub-topic. Energy level should alternate between high-intensity debate and calmer explanatory segments.
18. **Act 3 — Resolution (Chapter 6+):** Synthesis, future outlook, and takeaways. What does this mean for the listener? What should they think differently about now? End on a forward-looking, thought-provoking note.

4.1.2 Chapter Output Schema

Each chapter in the planner’s output must follow this structure (expressed as a JSON schema that the LLM must produce):

Field	Type	Description
chapter_number	int	Sequential chapter number (1-indexed)
title	string	Short, engaging chapter title (e.g., "The Diagnosis Revolution")
act	enum: setup/explore/resolve	Which act this chapter belongs to
energy_level	enum: high/medium/low	Target energy for pacing control
key_points	list[string]	3–5 key points this chapter must cover
source_chunk_ids	list[int]	IDs of source chunks to ground this chapter's content
transition_hook	string	One sentence that teases/connects to the next chapter
estimated_duration_minutes	float	Target length (total should sum to ~25–28 min, leaving room for intro/outro)

Pacing Controller (Improvement #2)

The energy_level field is the core of the Pacing Controller improvement. By explicitly tagging each chapter's energy level at the planning stage, we ensure the podcast has rhythm. The sequence should alternate: high → low → high → medium → high → low. If the Chapter Planner produces three consecutive "high" energy chapters, the validation step should flag this and request re-planning. Later, the SSML Annotator uses these energy tags to control speaking rate (faster for high energy, slower for low) and pause duration (shorter pauses for high energy, longer contemplative pauses for low).

4.2 Character Designer Agent

The Character Designer is one of the most important additions to the original plan. In the original sketch, characters were defined simply as "Host", "Participant 1", and "Participant 2" with basic system prompts. This is insufficient for natural dialogue because it produces characters that all sound the same — they all speak in the same register, use the same vocabulary, and react the same way.

The Character Designer agent creates rich, distinct personality profiles for each speaker. These profiles persist across all chapters, ensuring consistency. The key insight is that what makes a real podcast interesting is the contrast between the speakers — their different expertise, different communication styles, and different perspectives on the topic.

4.2.1 Character Persona Schema

Each character persona is a JSON object with the following fields:

Field	Type	Example
name	string	"Dr. Sarah Chen"
role	enum: host/expert/skeptic	"host"
expertise_area	string	"General science journalist, curious generalist"
speaking_style	string	"Uses analogies and metaphors heavily, asks clarifying questions, summarises complex points in simple terms"
vocabulary_level	enum: casual/moderate/technical	"casual" (for host), "technical" (for expert)
filler_patterns	list[string]	["you know", "like", "I mean", "right?"]
reaction_patterns	list[string]	["Oh interesting!", "Wait, really?", "Hmm, that's a good point"]
disagreement_style	string	"Politely challenges by asking probing questions rather than directly contradicting"
laugh_frequency	enum: rare/moderate/frequent	"moderate"
catchphrases	list[string]	["Let me push back on that", "OK so help me understand"]
emotional_range	string	"Gets genuinely excited about breakthroughs, skeptical of hype, empathetic about human impact"
tts_voice_id	string	"EXAVITQu4vr4xnSDxMaL" (ElevenLabs voice ID)

4.2.2 Role Archetypes

The system should create characters that fill complementary roles. The recommended configuration for a 3-speaker podcast is:

19. **The Host (Curious Generalist):** Asks the questions the listener would ask. Uses simple language. Summarises complex points. Keeps the conversation on track. Expresses surprise and curiosity authentically. This character is the listener's proxy.
20. **The Expert (Domain Authority):** Has deep knowledge of the topic. Uses technical terminology (which the Host then asks them to explain). Provides the substance. Tends to go deep and needs the Host to pull them back to accessibility.
21. **The Skeptic/Contrarian (Devil's Advocate):** Challenges assumptions. Brings up risks, downsides, and alternative viewpoints. Creates tension and debate. Without this role, the podcast becomes a one-sided lecture. This character makes it a conversation.

Why Not Just 2 Speakers?

Two speakers tend to fall into a monotonous Q&A pattern: host asks, expert answers, repeat. A third speaker (the skeptic) breaks this pattern by creating triangular dynamics — the expert and skeptic can debate while the host moderates, or the host and skeptic can gang up on the expert for a surprising claim. Three speakers also create more natural opportunities for interruptions and backchannels.

5. Phase 3: Dialogue Generation

Phase 3 is the heart of the system. It takes the chapter plans and character personas from Phase 2 and produces the actual dialogue script that will be spoken. This phase contains five agents that execute sequentially for each chapter. This is where the "genuine human conversation" magic happens, so we will cover each agent in detail.

5.1 Dialogue Engine

The Dialogue Engine generates the initial clean dialogue for a single chapter. It receives the chapter outline (key points, source chunks, energy level) and the character personas, and produces a multi-turn conversation script.

5.1.1 Beat Structure

Each chapter is internally structured into 5 "beats" — think of these as mini-scenes within the chapter. This structure ensures every chapter has a narrative arc of its own and doesn't just ramble. The beats are:

22. **Beat 1 — Hook (15–30 seconds):** The host opens with a provocative statement, surprising fact, or question that grabs attention. A guest might react with excitement or disbelief. Purpose: make the listener want to keep listening.
23. **Beat 2 — Context Building (60–90 seconds):** The host frames the topic for this chapter. The expert provides background. The host asks clarifying questions on behalf of the listener. Purpose: ensure the listener has enough context to follow the deep dive.
24. **Beat 3 — Deep Dive + Tension (90–120 seconds):** The meatiest part. The expert explains the core content. The skeptic challenges claims. The host mediates. There should be genuine back-and-forth, not just serial monologues. Purpose: deliver the key insights with intellectual tension.
25. **Beat 4 — Analogy / Aha Moment (30–60 seconds):** Someone (usually the expert) offers an analogy or simplification that crystallises the concept. The host has a visible "aha" reaction. Purpose: give the listener a memorable takeaway they can explain to someone else.
26. **Beat 5 — Wrap + Transition (15–30 seconds):** The host summarises the key point, then teases the next chapter. Purpose: create forward momentum.

5.1.2 Dialogue Output Schema

The Dialogue Engine outputs a structured JSON array of utterances:

Field	Type	Description
utterance_id	string	Unique ID (e.g., "ch1_u001")
speaker	string	Character name matching persona
beat	int	Which beat this utterance belongs to (1–5)
text	string	The actual dialogue text (clean, no markers yet)
intent	enum	question / answer / reaction / challenge / summary / transition
emotion	string	Dominant emotion: curious, excited, skeptical, thoughtful, amused
grounding_chunk_ids	list[int]	Source chunks this utterance draws from (for fact-checking)

5.1.3 Multi-Turn Generation Strategy

The Dialogue Engine does not generate the entire chapter script in a single LLM call. Instead, it generates beat-by-beat, with each beat's output appended to the conversation history before generating the next beat. This approach has three advantages:

- **Context continuity:** Each beat sees what was said in previous beats, so characters can reference earlier points naturally ("Going back to what you said about...").
- **Controllable pacing:** We can inject system instructions between beats to adjust energy level, switch who's leading the conversation, or force a topic transition.
- **Token management:** For a 30-minute podcast, the total script is ~4,500 words. Generating this all at once risks hitting output token limits. Beat-by-beat generation keeps each call's output to ~150–300 words.

5.2 Naturalness Injector Agent

This Is the Most Critical Agent in the Entire System

The Naturalness Injector is what separates a "podcast" from "two chatbots reading scripts". Without it, the output will sound polished but lifeless. Real humans do not speak in complete, grammatically perfect sentences. They hesitate, interrupt each other, laugh, say "um", start a sentence and then restart it differently, and make encouraging noises while someone else is talking. This agent adds all of those behaviours.

The Naturalness Injector takes the clean dialogue script from the Dialogue Engine and performs a second LLM pass to inject human speech patterns. It does NOT rewrite the content — it only adds markers and modifications that make the delivery feel natural.

5.2.1 Naturalness Markers (Complete Reference)

The agent inserts the following markers into the script. Each marker will later be interpreted by the SSML Annotator and Audio Overlap Engine:

Marker	Meaning	Example In Script	How TTS/Audio Handles It
[FILLER:thinking]	Insert a thinking filler before complex statement	"[FILLER:thinking] So basically, the way it works is..."	TTS renders "um" or "so" with a slight pause before
[FILLER:agreement]	Insert agreement filler	"[FILLER:agreement] that makes total sense"	TTS renders "yeah" or "right" with natural intonation
[INTERRUPT:0.3s]	Next speaker starts 0.3s before current speaker finishes	GUEST1: "...and that's why—" [INTERRUPT:0.3s] HOST: "Wait but doesn't that mean..."	Audio mixer overlaps the two clips by 0.3 seconds
[BACKCHANNEL]	Short agreement sound overlaid on active speaker	[BACKCHANNEL:HOST:"mm-hm"] during GUEST1's long explanation	Separate short TTS clip mixed at -8dB under active speaker
[FALSE_START]	Speaker starts a sentence, then restarts	"The thing about— well, let me put it differently..."	TTS renders both the false start and the correction naturally

Marker	Meaning	Example In Script	How TTS/Audio Handles It
[LAUGH:light]	Light chuckle	[LAUGH:light] "That's actually a really good analogy"	TTS voice with laughter quality, or pre-recorded laugh clip
[LAUGH:medium]	Genuine laugh	[LAUGH:medium] after a funny analogy	Slightly longer laugh, may include other speakers joining in
[EMPHASIS:word]	Prosodic stress on a specific word	"It's not about the [EMPHASIS:data], it's about the [EMPHASIS:interpretation]"	SSML <emphasis> tag on the marked word
[PAUSE:long]	Deliberate dramatic or thoughtful pause	"And that's when they realised... [PAUSE:long] ...it was completely wrong."	SSML <break time="800ms"/> inserted
[PACE:fast]	Speed up for excitement	"[PACE:fast] Oh my god yes exactly that's what I've been saying"	SSML <prosody rate="fast"> wrapper
[PACE:slow]	Slow down for emphasis	"[PACE:slow] And that... changes... everything."	SSML <prosody rate="slow"> wrapper

5.2.2 Injection Rules

The Naturalness Injector follows specific rules about when and how to inject markers. These rules are embedded in its system prompt:

27. **Filler frequency:** Insert thinking fillers before approximately 15–20% of utterances. Not every utterance — that sounds unnatural in the other direction. Characters with "casual" vocabulary_level get more fillers than "technical" characters.
28. **Interrupt frequency:** Approximately 10–15% of speaker transitions should be interruptions. The skeptic character interrupts more than others. Interrupts happen most during high-energy beats (Beat 3). Never interrupt during Beat 5 (wrap-up).
29. **Backchannel frequency:** During any utterance longer than 30 words, the host should produce at least one backchannel sound. Backchannels from other characters are rarer — about once per 50+ word monologue.
30. **False start rules:** False starts happen when a character is about to make a nuanced or controversial point. Maximum 2 false starts per chapter to avoid sounding glitchy.

31. **Laugh placement:** Laughs follow analogies, self-deprecating comments, absurd facts, or after genuine moments of agreement. The host laughs most frequently (if persona allows). Never place laughs after serious or sensitive content.
32. **Pacing markers:** Use [PACE:fast] when a character is genuinely excited or listing things quickly. Use [PACE:slow] for emphasis on key revelations or conclusions. Maximum 3–4 pace changes per chapter.

5.3 Fact-Checker Agent

The Fact-Checker is a quality gate that runs after the Naturalness Injector. Its job is to verify that every factual claim made in the dialogue can be traced back to one of the original source chunks from Phase 1. This is essential because LLMs will sometimes "improve" content by adding plausible-sounding but fabricated statistics, dates, or claims.

5.3.1 How It Works

33. **Claim extraction:** The agent reads each utterance and identifies specific factual claims (numbers, dates, named entities, causal assertions). Non-factual content (opinions, reactions, transitions) is skipped.
34. **Source matching:** For each claim, the agent checks the grounding_chunk_ids attached to that utterance. It retrieves those source chunks and uses an LLM call to determine whether the claim is "supported", "partially supported", or "unsupported" by the source material.
35. **Flagging:** Unsupported claims are flagged for rewrite. The system either removes the claim, replaces it with a hedge ("Some researchers suggest that..."), or routes the utterance back to the Dialogue Engine for regeneration with explicit instructions to stay grounded.

Cost Consideration

The Fact-Checker adds LLM calls per factual claim. For a 30-minute podcast, expect 50–100 factual claims. Using a fast model (Claude Haiku or GPT-4o-mini) for the claim-vs-source comparison keeps this step fast and cheap. Reserve the expensive model for the initial Dialogue Engine generation.

5.4 QA Reviewer Agent (Listener Simulator)

The QA Reviewer is the final quality gate before TTS. It reads the complete script (all chapters, after naturalness injection and fact-checking) from the perspective of a first-time listener and evaluates the following dimensions:

- **Repetition detection:** Are the speakers saying the same thing in different words across chapters? A common failure mode is Chapter 3 and Chapter 5 both explaining the same concept because the LLM didn't track what was already covered.
- **Engagement assessment:** Are there stretches longer than 90 seconds where only one speaker talks? These are "monologue traps" that lose listeners. Flag them for breaking into dialogue.
- **Clarity check:** Does the podcast assume knowledge that was never explained? The QA agent reads sequentially and flags any term or concept used before it was introduced.
- **Transition quality:** Does each chapter's end connect naturally to the next chapter's beginning? Abrupt topic switches lose listeners.
- **Energy arc validation:** Does the overall energy flow match the intended pacing? Three consecutive high-energy chapters or a low-energy ending should be flagged.

The QA Reviewer outputs a structured review with specific rewrite instructions. If issues are found, the affected chapters are routed back to the Dialogue Engine for targeted rewrites — not complete regeneration, just fixes to the flagged sections. This creates a refinement loop that can run 1–2 iterations maximum (to avoid infinite loops and cost explosion).

5.5 SSML Annotator Agent

The SSML (Speech Synthesis Markup Language) Annotator is the bridge between the text script and the audio generation. It converts the naturalness markers from the Naturalness Injector into actual SSML tags that the TTS engine can interpret. Different TTS providers support different subsets of SSML, so this agent must be configured for the specific provider being used.

5.5.1 Marker-to-SSML Mapping

Naturalness Marker	SSML Output (ElevenLabs)	SSML Output (Google Cloud TTS)
[FILLER:thinking]	Rendered as literal text: "um, " prepended	<break time="300ms"/>um,
[PAUSE:long]	<break time="800ms"/>	<break time="800ms"/>
[EMPHASIS:word]	Wrapped in <emphasis level="strong">	<emphasis level="strong">word</emphasis>
[PACE:fast]	<prosody rate="120%">...text...</prosody>	<prosody rate="fast">...text...</prosody>
[PACE:slow]	<prosody rate="80%">...text...</prosody>	<prosody rate="slow">...text...</prosody>
[LAUGH:light]	Rendered as "heh" + break (EL handles naturally)	<break time="200ms"/>
[INTERRUPT:0.3s]	Stripped (handled by audio mixer)	Stripped (handled by audio mixer)
[BACKCHANNEL]	Separate utterance object for mixing	Separate utterance object for mixing

Note that [INTERRUPT] and [BACKCHANNEL] markers are not converted to SSML. They are metadata that the Audio Overlap Engine (Phase 5) uses to mix audio tracks. The SSML Annotator strips these markers from the text and passes them as separate timing instructions to the mixer.

5.5.2 Per-Speaker Voice Configuration

The SSML Annotator also wraps each speaker’s utterances in voice-specific configuration. This includes the TTS voice ID (from the character persona), default speaking rate, and pitch. For example, the host might speak at a slightly faster base rate with higher pitch (conveying energy), while the expert speaks slower with lower pitch (conveying authority).

6. Phase 4: Voice Synthesis

Phase 4 converts the SSML-annotated script into raw audio files. Each utterance is sent to the TTS provider as an individual API call, producing one WAV file per utterance. This granularity is essential because the Audio Overlap Engine needs to manipulate individual utterance timings.

6.1 TTS Provider Comparison

Provider	Naturalness	SSML Support	Cost (per 1M chars)	Best For
ElevenLabs	Excellent — best conversational voices	Partial (breaks, emphasis, prosody)	~$11–30 depending on plan	Primary recommendation for this project
PlayHT v2	Very good — emotion control	Full SSML	~$10–20	Good alternative if ElevenLabs is unavailable
Google Cloud TTS (WaveNet/Neural2)	Good — but slightly robotic	Full SSML	~$16 (Neural2)	Budget option with excellent SSML control
Amazon Polly (Neural)	Moderate	Full SSML	~$4	Cheapest option, but least natural
OpenAI TTS (gpt-4o-audio)	Excellent naturalness	No SSML	~$15	Great quality but no fine-grained control

6.2 Cost Estimation for 30-Minute Podcast

A 30-minute podcast contains approximately 4,500 words or roughly 25,000–30,000 characters. At ElevenLabs' rates, this costs approximately $0.30–$0.90 per podcast in TTS alone. The LLM costs for the full pipeline (query generation, chapter planning, dialogue generation across 6 chapters, naturalness injection, fact-checking, QA review, and SSML annotation) add approximately $0.50–$2.00 depending on model choices. Total per-podcast cost: roughly $1–$3 which is very reasonable for high-quality output.

6.3 Voice Assignment Strategy

Each character gets a distinct TTS voice that remains consistent throughout the podcast. The voices should be selected for contrast: different genders, different accents, different pitch ranges. ElevenLabs offers voice cloning if specific voice characteristics are needed, or you can use their pre-built conversational voices. The key is that a listener should be able to tell which speaker is talking even without explicit naming — just from the voice quality.

7. Phase 5: Audio Post-Processing

Phase 5 takes the raw audio utterances from TTS and transforms them into a polished, professional-sounding podcast. This is a pure audio DSP (Digital Signal Processing) phase — no LLMs are involved. The tools used are pydub (Python audio manipulation library) and ffmpeg (command-line audio processing).

7.1 Audio Overlap Engine

This is the component that makes the podcast sound like a real conversation rather than people taking neat turns. In a real podcast, speakers overlap constantly: one person starts their response before the previous speaker has completely finished, someone says "mm-hm" while another person is mid-sentence, and laughter can overlap with speech.

7.1.1 How Overlapping Works

The Audio Overlap Engine reads the script's timing markers and constructs a multi-track audio timeline. Here is the step-by-step process:

36. **Sequential baseline:** Start by placing all utterances sequentially on a timeline, with a small gap (200–400ms) between speaker turns. This is the "robotic" baseline.
37. **Apply [INTERRUPT] markers:** For each interrupt marker (e.g., [INTERRUPT:0.3s]), shift the next utterance's start time backwards by the specified duration (300ms in this case) so it overlaps with the end of the previous utterance. Use pydub's overlay() method to mix the overlapping audio. Apply a slight volume reduction (-3dB) to the tail of the interrupted speaker's utterance so the interrupter is clearly audible.
38. **Apply [BACKCHANNEL] markers:** Backchannel sounds ("mm-hm", "right", "yeah") are generated as separate short TTS clips. Place these on a secondary audio track at the timestamp specified in the marker, mixed at -8dB relative to the main speaker. This creates the effect of someone making agreement sounds in the background while someone else is talking.
39. **Apply [LAUGH] markers:** Laughter is tricky for TTS. Two approaches: (a) Use TTS with laughter prompting (ElevenLabs handles this reasonably well with text like "hehe" or "haha" in the script), or (b) use pre-recorded laughter sound effects from a royalty-free

library, pitch-shifted to match the character's voice range. Mix laughter at -4dB relative to surrounding speech.

40. **Cross-fade transitions:** Between utterances from different speakers, apply a 50–100ms cross-fade to avoid hard audio cuts. This smooths the transition and reduces the "clicking" artefact that sometimes occurs at audio segment boundaries.

pydub Example: Overlapping Two Audio Clips

```
from pydub import AudioSegmenthost_clip =
AudioSegment.from_wav("host_utterance_12.wav")guest_clip =
AudioSegment.from_wav("guest_utterance_13.wav")# Overlap: guest starts 300ms before host
finishesoverlap_ms = 300host_end = host_clip[-overlap_ms:] - 3  # reduce host volume in
overlap regionmixed_overlap = host_end.overlay(guest_clip[:overlap_ms])result =
host_clip[:-overlap_ms] + mixed_overlap + guest_clip[overlap_ms:]
```

7.2 Audio Post-Processing

After the overlap engine produces mixed chapter audio, several post-processing steps are applied to achieve professional podcast quality:

41. **Noise gate:** Remove very quiet noise between utterances (below -40dB). TTS output is generally clean, but some providers introduce subtle artifacts in silence regions.
42. **Equalisation (EQ):** Apply a gentle voice-optimised EQ curve: slight boost at 2–5 kHz (presence/clarity), slight cut below 100 Hz (rumble removal). Use ffmpeg's equalizer filter.
43. **Dynamic compression:** Apply light compression (ratio 2:1, threshold -20dB) to even out volume differences between speakers and between quiet and loud passages. This is standard podcast mastering.
44. **Loudness normalisation:** Normalise to -16 LUFS (the standard loudness for podcasts, per Apple Podcasts and Spotify guidelines). Use ffmpeg's loudnorm filter.
45. **Ambient room tone (Improvement #4):** Add a very subtle room tone or ambient background at -30dB to -35dB. This fills the "digital silence" between utterances that makes TTS output sound unnatural. Options include: light coffee shop ambience, soft room tone, or gentle background music during transitions. Use royalty-free sources.

7.3 Cold Open / Teaser Generator (Improvement #3)

Every professional podcast begins with a "cold open" — a 15–30 second teaser played before the intro music. This is a short, attention-grabbing clip pulled from later in the episode that makes the listener want to keep listening.

7.3.1 How It Works

46. **Script scanning:** An LLM agent reads the complete final script and identifies the 2–3 most compelling moments. Compelling moments include: surprising revelations, strong disagreements between speakers, emotionally resonant analogies, or moments of genuine laughter/excitement.
47. **Teaser assembly:** The agent selects the best moment and extracts a short excerpt (2–4 utterances, ~15–20 seconds of audio). It adds a framing line like: "Later in this episode..." before the excerpt and "Stay tuned" or a transition sound after.
48. **Audio extraction:** Since the full chapter audio is already generated at this point, the cold open is literally extracted from the existing chapter audio by timestamp. No additional TTS calls needed — just audio slicing with pydub.

7.4 Chapter Stitcher

The final step assembles all the pieces into a single podcast MP3 file. The assembly order is:

49. **Cold open teaser** (15–30 seconds)
50. **Intro music / jingle** (5–10 seconds, royalty-free, fades in under the cold open's last 3 seconds)
51. **Host introduction** ("Welcome to [Podcast Name], I'm your host...", ~15 seconds)
52. **Chapters 1 through N** (with 1–2 second transition sounds or brief music stings between chapters)
53. **Host outro** ("Thanks for listening...", ~15 seconds)
54. **Outro music** (fades in under host's last sentence, plays for 5–10 seconds)

The stitcher applies final loudness normalisation to the complete file and exports as MP3 at 128kbps CBR (standard podcast bitrate) or 192kbps for higher quality. Metadata tags (title, description, episode number) are embedded in the MP3 ID3 tags.

8. Consolidated Improvements Summary

This section provides a consolidated reference of all 8 improvements over the original design, their purpose, and where they fit in the pipeline.

#	Improvement	Phase	Purpose	Complexity
1	Dedup + Relevance Scorer	Phase 1	Remove noise and redundancy from source material; ensure only high-quality, relevant content reaches the LLM	Medium — requires embedding model + FAISS + cross-encoder
2	Pacing Controller	Phase 2	Ensure the podcast has rhythmic energy variation; prevent listener fatigue from constant intensity	Low — metadata field on chapter plans + SSML rate mapping
3	Cold Open / Teaser Generator	Phase 5	Hook listeners in the first 15 seconds with the most compelling moment from the episode	Low — LLM scan + audio slice
4	Ambient Sound Designer	Phase 5	Fill "digital silence" with subtle room tone to make TTS output sound like a real recording	Low — pydub overlay of background audio at -30dB
5	Iterative Refinement Loop (QA Reviewer)	Phase 3	Catch repetition, monologue traps, unclear explanations, and broken transitions before TTS	Medium — full-script LLM review + targeted rewrite loop
6	Character Designer Agent	Phase 2	Create rich, distinct speaker personas that produce contrasting dialogue styles	Medium — requires persona schema design + integration into every dialogue call
7	Naturalness Injector Agent	Phase 3	Add fillers, interruptions, false starts, backchannels, and laughter to make dialogue feel human	High — core differentiator, requires careful prompt engineering + audio mixer integration
8	SSML Annotator Agent	Phase 3	Convert naturalness markers into TTS-interpretable SSML tags for prosody control	Medium — provider-specific SSML mapping + voice configuration

9. Timing and Sizing Calculations

Understanding the math behind podcast duration is essential for configuring the Chapter Planner and Dialogue Engine correctly. Here are the key numbers:

Metric	Value	How Derived
Average speaking rate	150 words per minute	Industry standard for conversational speech
Target podcast duration	25–28 minutes (content)	30 min total minus intro/outro/transitions
Total word count	3,750–4,200 words	25–28 min × 150 wpm
Number of chapters	6–8	Each chapter = 3–5 minutes
Words per chapter	450–700 words	3–5 min × 150 wpm
Beats per chapter	5	Fixed structure (hook, context, deep dive, aha, wrap)
Words per beat	90–140 words	Chapter words ÷ 5 beats
LLM calls per chapter (dialogue)	5	One per beat (sequential generation)
Total LLM calls for dialogue	30–40 for 6 chapters	5 beats × 6 chapters
TTS utterances per chapter	20–40	Multi-speaker dialogue has many short turns
Total TTS API calls	120–320	20–40 × 6–8 chapters

10. Next Steps

With this design document as the foundation, the next phases of the project are:

55. **Framework Selection:** Choose the agent orchestration framework. LangGraph is the recommended choice given the multi-agent pipeline structure, checkpoint requirements, and conditional routing needed for the QA refinement loop.
56. **Graph Topology Design:** Map each agent/node from this document to a LangGraph node. Design the edges (including conditional edges for retry loops), state schema, and checkpoint strategy.
57. **Prototype Phase 1–2:** Build and test the Research + Content Planning pipeline end-to-end with a sample topic. Validate that source chunks are high-quality and chapter plans are coherent.
58. **Prototype Phase 3:** Build the Dialogue Engine and Naturalness Injector. This is the highest-risk component — iterate on prompt engineering until the dialogue sounds genuinely conversational when read aloud.
59. **TTS Integration:** Connect to ElevenLabs (or chosen provider), test SSML rendering, and build the Audio Overlap Engine.
60. **End-to-End Pipeline:** Connect all phases, run a full generation, and listen critically to the output. Iterate on quality.
61. **Cloud Deployment:** Containerise the pipeline and deploy to AWS (Lambda for lightweight steps, ECS/Fargate for TTS-heavy steps, S3 for audio storage).

Ready to Architect

This document covers the what and why. The next document will cover the how: LangGraph node topology, state schema, Pydantic models, FastAPI entry point, and AWS deployment architecture.